```
#!/usr/bin/env python3
r"""
supt_resonance.py — live Resonance Score for the WolfWatch Sentinel.

Turns the Sentinel's existing feeds into d_ij addresses on the same axis as the
rest of the SUPT corpus, using the frozen probe unmodified.

WHY THIS WORKS
    The operator takes any ordered positive sequence and returns one scalar.
    The Sentinel already pulls sequences: earthquake interevent times, magnitudes
    in time order, planetary K index, solar wind speed. Those are valid inputs
    with no adaptation needed. The score is therefore directly comparable to
    every other entry in the corpus — ribosome 1.88, tokamak 1.93, zeta floor
    3.6125 — rather than being a bespoke number.

DESIGN NOTES
    - The probe is frozen: alpha = 0.01, unchanged since 2025-01-20. Do not tune
    - Every score ships with a shuffle null so the reading has context. A d_ij
      that matches its own shuffle is a null, and should display as one.
    - Live windows are short. N < 50 forces the tail rule to span nearly the whol
      accumulator; reads stay regime-valid but lose decimal precision. The module
      reports N and flags short windows rather than hiding them.
    - Null is a permitted outcome. Display it.

USAGE (as a module)
    from supt_resonance import resonance_score, usgs_sequences, swpc_sequences

    for name, seq in usgs_sequences(period="day", minmag=2.5).items():
        print(name, resonance_score(seq))

Requires: numpy, requests
"""
from __future__ import annotations
import numpy as np

# ======================================================================
# THE FROZEN PROBE — alpha = 0.01, frozen 2025-01-20. DO NOT MODIFY.
# ======================================================================
ALPHA = 0.01

def supt_probe(x) -> float | None:
    x = np.asarray(x, dtype=float)
    x = x[np.isfinite(x)]
    if x.size < 4:
```

```
        return None
    med = np.median(x)
    mad = np.median(np.abs(x - med))
    x = (x - med) / (mad + 1e-12)
    phi = np.cumsum(x)
    g = np.diff(phi)
    g = g / (np.mean(np.abs(g)) + 1e-12)
    C = np.zeros(len(g))
    C[0] = np.cos(2 * np.pi * g[0])
    for i in range(1, len(g)):
        C[i] = ALPHA * np.cos(2 * np.pi * g[i]) + (1 - ALPHA) * C[i - 1]
    tail = max(50, int(0.2 * len(C)))
    return float(-np.log(np.mean(np.abs(C[-tail:])) + 1e-12))

BANDS = [(1.0, "COHERENCE"), (2.0, "CLUTCH"), (3.6125, "SUB-FLOOR")]

def band(d: float | None) -> str:
    if d is None:
        return "N/A"
    for edge, name in BANDS:
        if d < edge:
            return name
    return "VACUUM"

# reference anchors from the corpus, for display context
ANCHORS = {
    "zeta vacuum floor": 3.6125,
    "CLUTCH cusp": (1.88, 1.96),
    "ribosome translocation": 1.88,
    "tokamak L-H": 1.93,
    "CLASH lensing masses": 1.9102,
    "ACCEPT X-ray T": 1.3702,
}

# ============================================================================
# SCORING
# ============================================================================
def resonance_score(values, n_shuffle: int = 200, seed: int = 20250120) -> dict:
    """
    Score an ordered positive sequence.

    Returns a dict ready for a UI:
        d_ij, band, n, null_mean, null_sd, z, separated, short_window, note
    """
    v = np.asarray(values, dtype=float)
    v = v[np.isfinite(v)]
    d = supt_probe(v)
```

```
    if d is None:
        return {"d_ij": None, "band": "N/A", "n": int(v.size),
                "note": "sequence too short to score (need >= 4)"}

    rng = np.random.default_rng(seed)
    null = np.array([supt_probe(rng.permutation(v)) for _ in range(n_shuffle)],
                    dtype=float)
    null = null[np.isfinite(null)]
    nm, ns = float(null.mean()), float(null.std())
    z = (d - nm) / (ns + 1e-12)

    short = v.size < 50
    note = []
    if short:
        note.append("short window: N < 50, tail rule spans nearly the whole "
                    "accumulator; regime-valid, low decimal precision")
    if abs(z) < 2:
        note.append("not separated from shuffle — read as null")
    if 1.88 <= d <= 1.96:
        note.append("in the CLUTCH cusp; check whether the source distribution "
                    "is heavy-tailed, which can reach this band on its own")

    return {
        "d_ij": round(d, 4),
        "band": band(d),
        "n": int(v.size),
        "null_mean": round(nm, 4),
        "null_sd": round(ns, 4),
        "z": round(float(z), 2),
        "separated": bool(abs(z) >= 3),
        "short_window": short,
        "note": "; ".join(note) if note else "",
    }

# ============================================================================
# FEED ADAPTERS
# ============================================================================
USGS = ("https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/"
        "{mag}_{period}.geojson")
SWPC_KP = "https://services.swpc.noaa.gov/products/noaa-planetary-k-index.json"
SWPC_WIND = ("https://services.swpc.noaa.gov/products/solar-wind/"
             "plasma-1-day.json")

def usgs_sequences(period: str = "day", mag: str = "2.5") -> dict:
    """
    period: 'hour' | 'day' | 'week' | 'month'
    mag:    'significant' | '4.5' | '2.5' | '1.0' | 'all'
```

```
    Returns the natural ordered sequences from the catalogue:
      interevent_times : gaps between successive origin times, seconds
      magnitudes       : magnitudes in time order (shifted positive)
      depths           : depths in time order, km
    """
    import requests
    url = USGS.format(mag=mag, period=period)
    js = requests.get(url, timeout=30).json()
    feats = js.get("features", [])
    rows = []
    for f in feats:
        p = f.get("properties", {})
        g = f.get("geometry", {}).get("coordinates", [None, None, None])
        if p.get("time") is None:
            continue
        rows.append((p["time"], p.get("mag"), g[2] if len(g) > 2 else None))
    rows.sort(key=lambda r: r[0])
    if len(rows) < 5:
        return {}
    t = np.array([r[0] for r in rows], dtype=float) / 1000.0   # ms -> s
    m = np.array([r[1] if r[1] is not None else np.nan for r in rows], dtype=floa
    z = np.array([r[2] if r[2] is not None else np.nan for r in rows], dtype=floa

    inter = np.diff(t)
    inter = inter[inter > 0]
    # magnitudes can be negative; shift to positive without changing the gaps
    mm = m[np.isfinite(m)]
    mm = mm - np.nanmin(mm) + 1e-3 if mm.size else mm
    zz = z[np.isfinite(z)]
    zz = zz[zz > 0]

    out = {}
    if inter.size >= 4: out["usgs_interevent_times"] = inter
    if mm.size    >= 4: out["usgs_magnitudes"]       = mm
    if zz.size    >= 4: out["usgs_depths"]           = zz
    return out

def swpc_sequences() -> dict:
    """
    Planetary K index and solar wind plasma, in time order.
    Both SWPC endpoints return [header_row, *data_rows].
    """
    import requests
    out = {}
    try:
        kp = requests.get(SWPC_KP, timeout=30).json()
```

```
        hdr, rows = kp[0], kp[1:]
        i = hdr.index("Kp") if "Kp" in hdr else 1
        v = np.array([float(r[i]) for r in rows if r[i] not in (None, "")],
                     dtype=float)
        v = v + 1e-3                       # Kp can be 0; keep strictly positive
        if v.size >= 4: out["swpc_kp"] = v
    except Exception as e:
        out["_kp_error"] = str(e)
    try:
        pw = requests.get(SWPC_WIND, timeout=30).json()
        hdr, rows = pw[0], pw[1:]
        for col, key in (("speed", "swpc_wind_speed"),
                         ("density", "swpc_wind_density")):
            if col in hdr:
                i = hdr.index(col)
                v = np.array([float(r[i]) for r in rows
                              if r[i] not in (None, "")], dtype=float)
                v = v[v > 0]
                if v.size >= 4: out[key] = v
    except Exception as e:
        out["_wind_error"] = str(e)
    return out

# ============================================================================
# CLI
# ============================================================================
def main():
    print("=" * 74)
    print("SUPT Resonance Score — frozen probe, alpha = 0.01")
    print("Null is a permitted outcome. Report what comes back.")
    print("=" * 74)

    for label, fn in (("USGS earthquake catalogue", lambda: usgs_sequences("day",
                      ("NOAA SWPC space weather", swpc_sequences)):
        print(f"\n--- {label} ---")
        try:
            seqs = fn()
        except Exception as e:
            print(f"  feed error: {e}")
            continue
        if not seqs:
            print("  no sequences returned (window may be empty)")
            continue
        for name, seq in seqs.items():
            if name.startswith("_"):
                print(f"  {name}: {seq}")
                continue
```

```
            r = resonance_score(seq)
            flag = " *" if r.get("separated") else ""
            print(f"  {name:26s} N={r['n']:5d}  d_ij={r['d_ij']}  "
                  f"{r['band']:10s} z={r.get('z')}{flag}")
            if r.get("note"):
                print(f"      note: {r['note']}")

    print("\ncorpus anchors for comparison:")
    for k, v in ANCHORS.items():
        print(f"  {k:26s} {v}")

if __name__ == "__main__":
    main()
```